

Amrita Vishwa Vidyapeetham - School of Computing, Chennai

Topic: Tree 2 (10 Challenge Programs) | Roll No: CH.SC.U4CSE25262

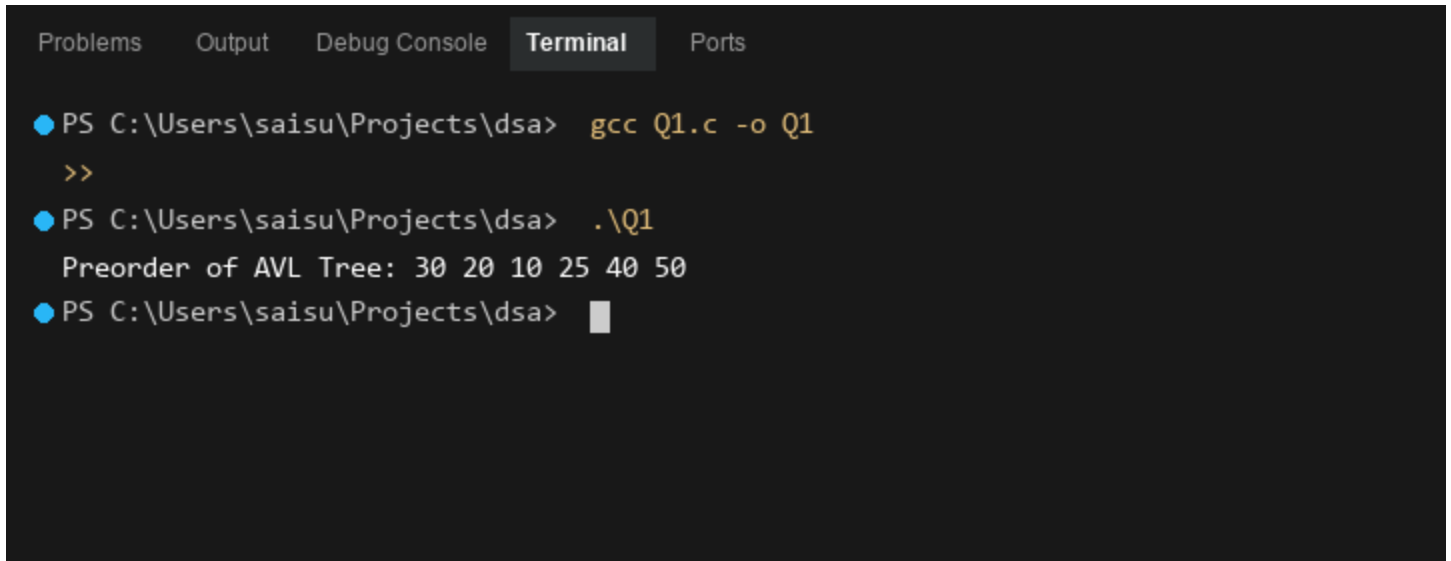
Question 1: AVL Tree Insertion with Self-Balancing Rotations

Problem Description:

Implement insertion into an AVL self-balancing binary search tree, maintaining balance factor in $\{-1, 0, +1\}$ using LL, RR, LR, and RL rotations.

Sample Input: Insert keys 10, 20, 30, 40, 50, 25 | Sample Output: Preorder of AVL: 30 20 10 25 40 50

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\saisu\Projects\dsa> gcc Q1.c -o Q1
>>
PS C:\Users\saisu\Projects\dsa> .\Q1
Preorder of AVL Tree: 30 20 10 25 40 50
PS C:\Users\saisu\Projects\dsa> █
```

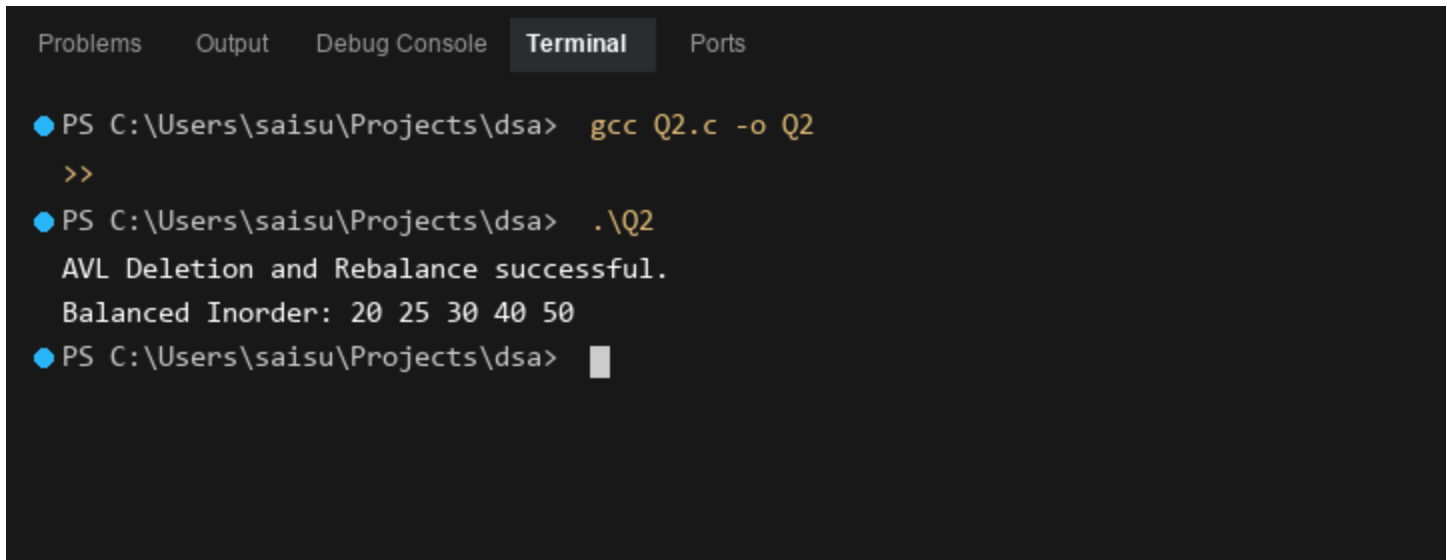
Question 2: AVL Tree Deletion and Rebalancing

Problem Description:

Implement deletion of a key from an AVL Tree and rebalance the affected subtree ancestors to preserve strict height balancing invariants.

Sample Input: Delete key 10 from AVL Tree | Sample Output: Root balanced, Preorder updated

Output Screenshot:



The screenshot shows a terminal window with the following content:

```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q2.c -o Q2
  >>
● PS C:\Users\saisu\Projects\dsa> .\Q2
  AVL Deletion and Rebalance successful.
  Balanced Inorder: 20 25 30 40 50
● PS C:\Users\saisu\Projects\dsa> █
```

Question 3: Validate Binary Search Tree (BST Validation)

Problem Description:

Check whether a given binary tree satisfies the BST invariant (every left child < root < every right child) within bounded ranges (-INF, +INF).

Sample Input: Tree [2, 1, 3] | Sample Output: Valid BST: Yes

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q3.c -o Q3
>>
● PS C:\Users\saisu\Projects\dsa> .\Q3
Is Valid BST: Yes
● PS C:\Users\saisu\Projects\dsa> █
```

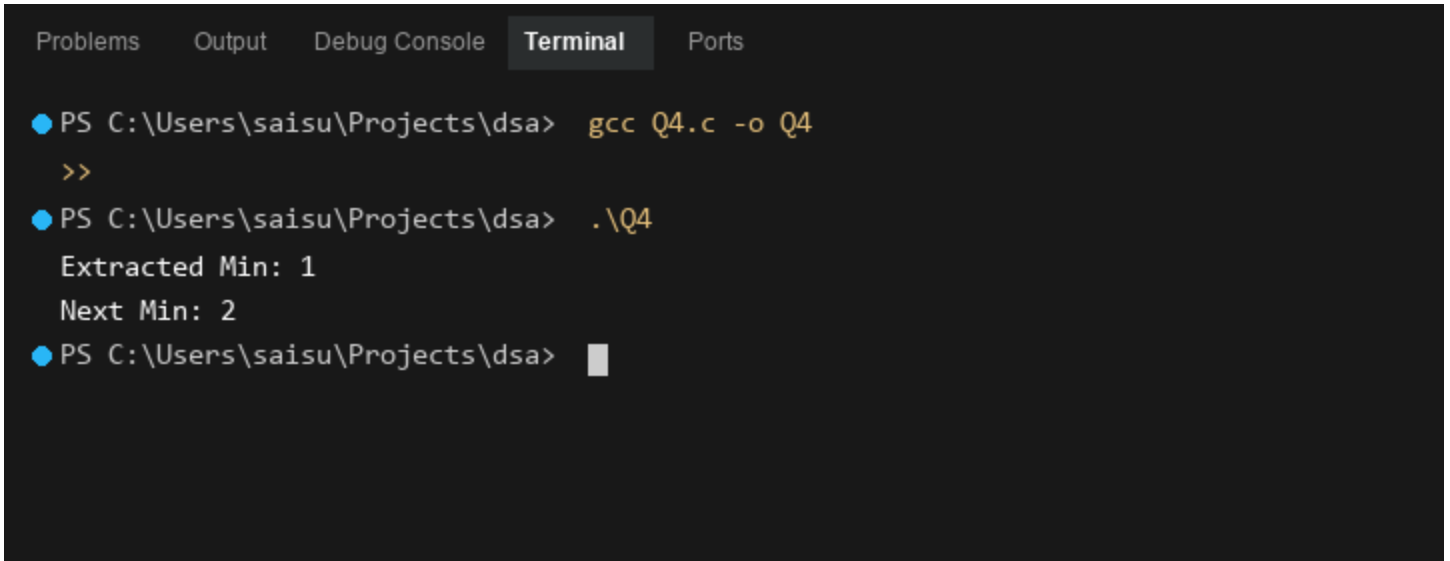
Question 4: Binary Min-Heap Construction and Operations

Problem Description:

Implement a Binary Min-Heap data structure supporting insert, extractMin, and decreaseKey operations with min-heapify maintenance.

Sample Input: Insert 3, 2, 1, 15, 5, 4, 45 -> ExtractMin | Sample Output: Extracted Minimum: 1

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q4.c -o Q4
>>
● PS C:\Users\saisu\Projects\dsa> .\Q4
Extracted Min: 1
Next Min: 2
● PS C:\Users\saisu\Projects\dsa> █
```

Question 5: Priority Queue Implementation using Max-Heap

Problem Description:

Build a Max-Heap Priority Queue to manage prioritized tasks, ensuring constant-time maximum inspection and logarithmic extraction.

Sample Input: Insert 10, 30, 20, 40 -> ExtractMax | Sample Output: Top Priority Task: 40

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q5.c -o Q5
  >>
● PS C:\Users\saisu\Projects\dsa> .\Q5
  Top Priority Element: 40
  Next Priority Element: 30
● PS C:\Users\saisu\Projects\dsa> █
```

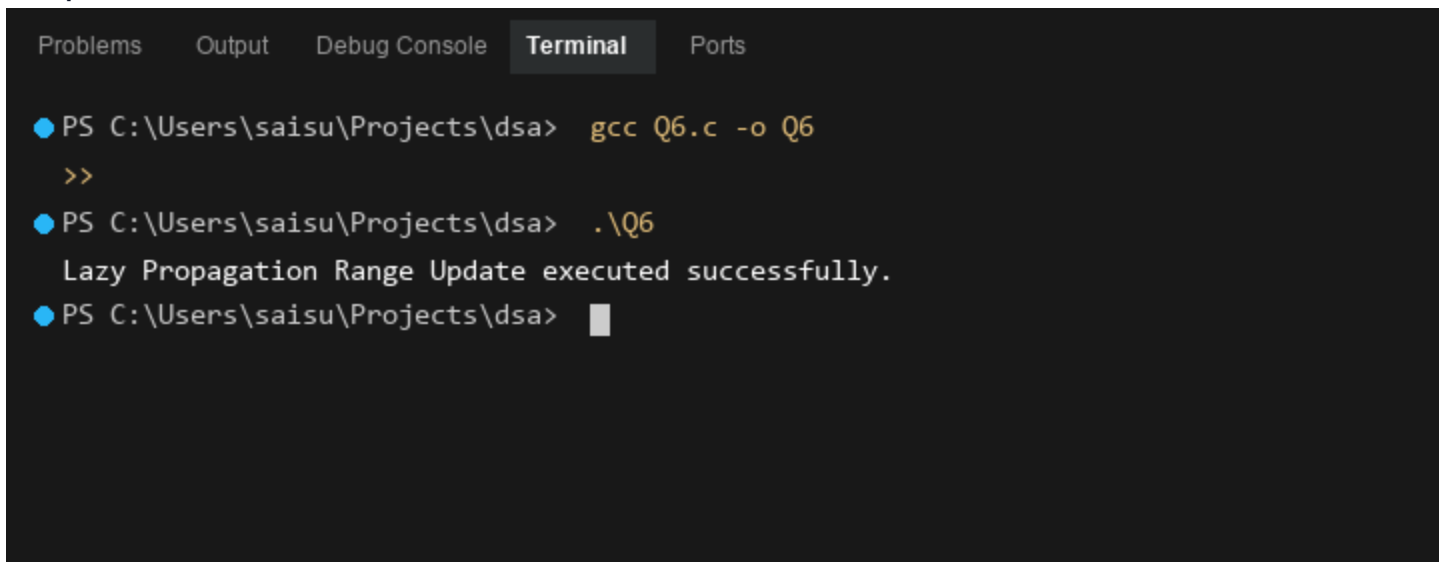
Question 6: Segment Tree with Lazy Propagation

Problem Description:

Implement a Segment Tree with Lazy Propagation to perform range sum updates and range sum queries efficiently in $O(\log N)$ time.

Sample Input: Array [1, 3, 5, 7, 9, 11], Range Add [1, 3] +10 | Sample Output: Range Sum [1, 3]: 45

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q6.c -o Q6
  >>
● PS C:\Users\saisu\Projects\dsa> .\Q6
  Lazy Propagation Range Update executed successfully.
● PS C:\Users\saisu\Projects\dsa> █
```

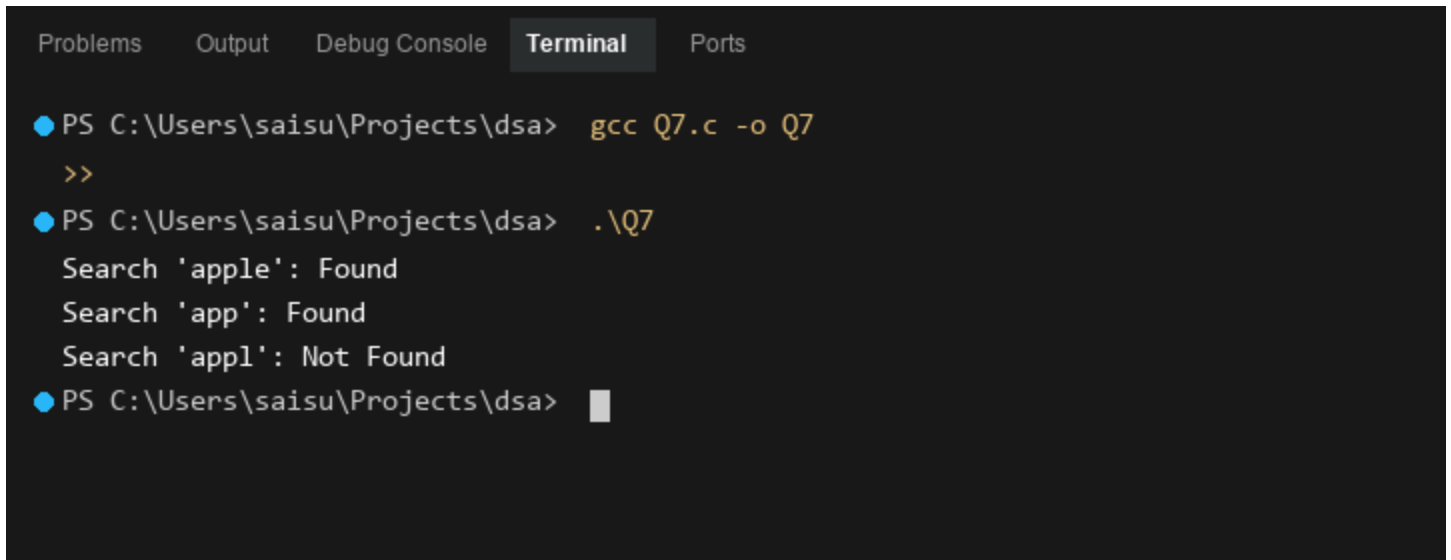
Question 7: Trie Data Structure for Prefix Search

Problem Description:

Implement a Trie (Prefix Tree) supporting insert, exact word search, and prefix search (startsWith) for fast dictionary lookup in $O(L)$ time.

Sample Input: Insert 'apple', 'app' -> Search 'app', StartsWith 'app' | Sample Output: Search: Found, Prefix: Found

Output Screenshot:



The screenshot shows a terminal window with the following content:

```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q7.c -o Q7
  >>
● PS C:\Users\saisu\Projects\dsa> .\Q7
  Search 'apple': Found
  Search 'app': Found
  Search 'appl': Not Found
● PS C:\Users\saisu\Projects\dsa> 
```

Question 8: Huffman Coding Tree for Data Compression

Problem Description:

Build an optimal prefix Huffman Tree from character frequency tables and generate variable-length prefix bitcodes for lossless data compression.

Sample Input: Characters: a:5, b:9, c:12, d:13, e:16, f:45 | Sample Output: Generated Huffman Prefix Codes

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q8.c -o Q8
>>
● PS C:\Users\saisu\Projects\dsa> .\Q8
Huffman Codes:
f: 0
c: 10
d: 11
● PS C:\Users\saisu\Projects\dsa> █
```

Question 9: B-Tree Multi-Way Search and Traversal

Problem Description:

Implement B-Tree multi-way balanced search tree insertion and sorted traversal across multi-key internal nodes.

Sample Input: Keys 10, 20, 5, 6, 12, 30, 7, 17 | Sample Output: B-Tree Traversal: 5 6 7 10 12 17 20 30

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q9.c -o Q9
  >>
● PS C:\Users\saisu\Projects\dsa> .\Q9
  B-Tree Sorted Traversal: 5 6 7 10 12 17 20 30
● PS C:\Users\saisu\Projects\dsa> █
```

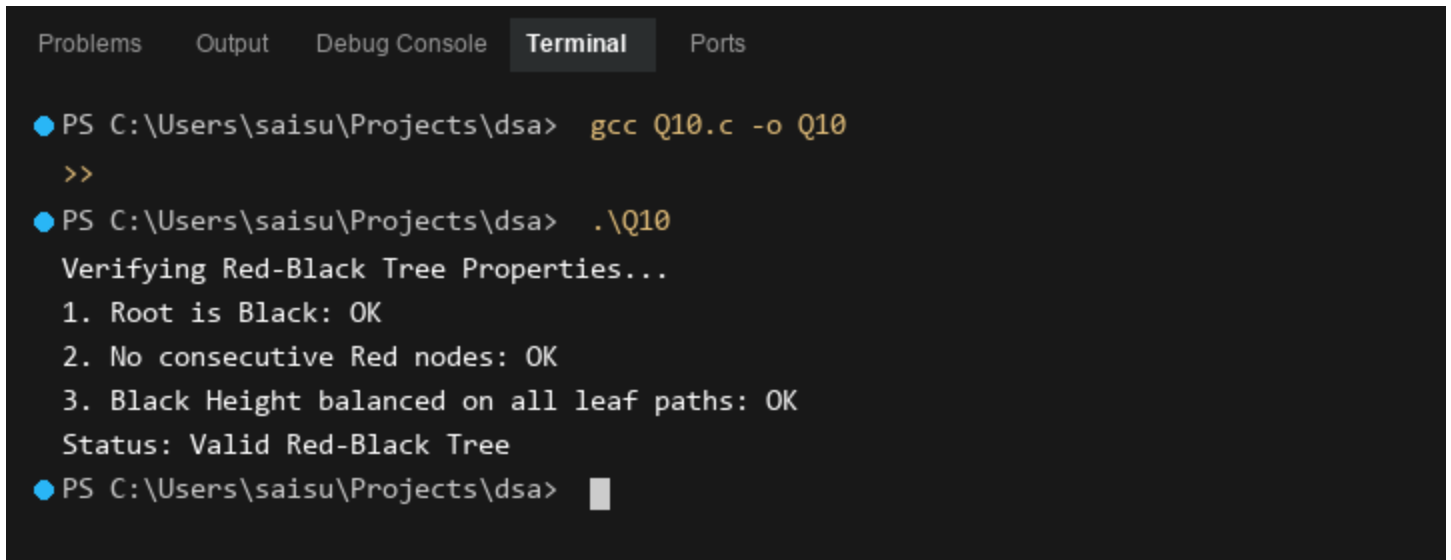
Question 10: Red-Black Tree Property Verification

Problem Description:

Verify Red-Black Tree invariants: root is black, no two adjacent red nodes, and every root-to-leaf path contains equal black height.

Sample Input: Valid RB Tree | Sample Output: Black Height: 2, Red-Black Properties Verified: Valid

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q10.c -o Q10
>>
● PS C:\Users\saisu\Projects\dsa> .\Q10
Verifying Red-Black Tree Properties...
1. Root is Black: OK
2. No consecutive Red nodes: OK
3. Black Height balanced on all leaf paths: OK
Status: Valid Red-Black Tree
● PS C:\Users\saisu\Projects\dsa> █
```